

Mate al profe



**INSTITUCIÓN UNIVERSITARIA
COLEGIO MAYOR DEL CAUCA**

Presentado por:

Ronaldo Basto Pillimur

Oscar Antonio Solarte

Cristóbal Alexander Chantre Vernaza

Presentado a:

Marta Cecilia Camacho Ojeda

Materia: Construcción de Software

Institución Universitaria Colegio Mayor del Cauca

Facultad de Ingeniería

Ingeniería Informática

Popayán-Cauca

2026

1. Descripción del producto software

Mate al Profe es un videojuego desarrollado en Java como aplicación de consola, cuyo propósito es ofrecer una experiencia de juego sencilla basada en enfrentamientos entre el jugador y diferentes profesores.

El jugador puede seleccionar un profesor de un repertorio disponible, escoger un arma y seleccionar una zona de ataque. Dependiendo de las características del profesor, el arma seleccionada y la zona elegida, el sistema determina el resultado de cada ataque y actualiza el estado de la partida.

El proyecto se desarrolla aplicando principios de programación orientada a objetos y patrones de diseño, con especial énfasis en Factory Method, Builder y Singleton.

La aplicación cuenta con una interfaz de consola, debido a que el objetivo principal del proyecto es demostrar la correcta aplicación de los patrones de diseño y la organización del software.

El sistema busca demostrar cómo los patrones de diseño pueden utilizarse para construir un programa más organizado, reutilizable, mantenible y fácil de ampliar.

Objetivo general

Desarrollar una aplicación de consola en Java denominada “Mate al Profe”, aplicando programación orientada a objetos y patrones de diseño para gestionar la creación de personajes, armas y comportamientos del juego de manera organizada, reutilizable y escalable.

Objetivos específicos

- Diseñar una aplicación de consola que permita al usuario iniciar y desarrollar una partida.
- Implementar diferentes tipos de profesores y armas mediante programación orientada a objetos.
- Aplicar el patrón Factory Method para crear diferentes tipos de profesores.
- Aplicar el patrón Singleton para controlar la información principal de la partida.
- Aplicar el patrón Builder para construir los objetos Profesor.
- Diseñar una estructura de clases coherente con los patrones seleccionados.
- Implementar y probar el sistema utilizando Java.
- Documentar las decisiones de diseño y justificar el uso de cada patrón.

2. Pila del Producto: Historias de usuario

HU - 001 Iniciar una partida

Como jugador.

Quiero iniciar una partida nueva.

Para comenzar un enfrentamiento contra un profesor.

Criterios de Aceptación:

- El menú principal muestra la opción de “iniciar partida”.
- Al elegirla, se crea una partida nueva desde cero.
- El sistema debe mostrar el listado de profesores disponibles.

Prioridad: Alta.

HU - 002 Selección de Profesor

Como jugador.

Quiero elegir a qué profesor voy a matar.

Para medirme contra él con una salud y una dificultad distintas.

Criterios de Aceptación:

- En el listado se muestra el nombre, materia y salud máxima del profesor antes de confirmarlo.
- El profesor elegido queda asignado a la partida en curso.

Prioridad: Alta

HU - 003 Selección de Arma

Como jugador.

Quiero elegir un arma dentro del repertorio disponible.

Para decidir cuánto daño puedo causar y que tan preciso va a ser el ataque.

Criterios de Aceptación:

- El catálogo debe mostrar las armas disponibles.
- Cada arma debe mostrar su daño base y precisión.
- El arma seleccionada debe quedar disponible para realizar el ataque.

Prioridad: Media.

HU - 004 Seleccionar la Zona de Impacto

Como jugador.

Quiero elegir la zona del cuerpo a la que voy a apuntar.

Para asumir un nivel de riesgo a cambio de un multiplicador de daño.

Criterios de Aceptación:

- El sistema debe mostrar tres zonas: cabeza, torso y extremidades.
- Cada zona debe tener una probabilidad de acierto.
- Cada zona debe tener un multiplicador de daño.

Prioridad: Media.

HU - 005 Información de la Partida en Curso

Como jugador.

Quiero ver el estado actual de la partida después de cada acción

Para seguir la evolución del combate mientras juego.

Criterios de Aceptación:

- Después de cada ataque, la consola debe informar si fue acertado o fallido.
- Cuando el ataque sea exitoso, se debe mostrar el daño realizado.
- La vida restante del profesor debe actualizarse.
- Cuando la vida llegue a cero, el sistema debe informar el final de la partida.

Prioridad: Media

3. Pila del Sprint: Historias de Usuario y Criterios de Aceptación

Sprint 1 - Selección e inicialización

Historias: HU-001 y HU-002.

Criterios de Aceptación:

- El sistema debe mostrar un menú inicial para comenzar la partida.
- Se debe mostrar la lista de profesores disponibles.
- Se debe mostrar el nombre, materia y salud máxima.
- El sistema debe validar la opción ingresada.
- El profesor seleccionado debe ser asignado a la partida.

Sprint 2 - Mecánica de ataque y selección de equipamiento

Historias: HU-003 y HU-004.

Criterios de aceptación:

- El usuario debe poder seleccionar un arma mediante un índice numérico.
- El sistema debe mostrar las características del arma.
- El sistema debe mostrar las tres zonas disponibles.
- Cada zona debe tener una probabilidad y multiplicador diferente.
- El cálculo del ataque debe combinar la precisión del arma con la probabilidad de la zona.

Sprint 3 - Resultado del ataque y finalización

Historia: HU-005.

Criterios de aceptación:

- El sistema debe informar si el ataque acierta o falla.
- Cuando acierte, debe informar el daño realizado.
- La vida del profesor debe actualizarse.
- Cuando la vida llegue a cero, el sistema debe informar que la partida terminó.

4. Principios SOLID: ¿Cuál se cumple y cuál no cumple?

- **1. SRP — Responsabilidad Única** **Cumple parcialmente**

Este principio busca que cada clase tenga una responsabilidad principal. En el proyecto esto se cumple en varias clases. Por ejemplo, Arma se encarga únicamente de almacenar las características del arma:

```
1 public class Arma{
2     private String nombre;
3     private int golpebase;
4     private double precision;
5 }
```

También Partida se encarga principalmente de controlar la lógica del ataque:

```
8     public void atacar(Arma arma, ZonaCuerpo zona) {
9         double probabilidadExito = arma.getPrecision() * zona.getProbabilidadAcierto();
```

Sin embargo, Main realiza varias tareas al mismo tiempo, como mostrar menús, recibir opciones, seleccionar profesores, armas y zonas y controlar el flujo de la partida.

- **2. OCP — Abierto/Cerrado** **Cumple**

Este principio indica que el sistema debe permitir agregar nuevas funcionalidades sin modificar demasiado el código existente.

En el proyecto se utiliza ProfesorFactory:

```
1 public abstract class ProfesorFactory {
2     public abstract Profesor crearProfesor();
3 }
```

Esto permite crear nuevas fábricas, por ejemplo:

```
1  public class ProfesorFisicaFactory extends ProfesorFactory {
2      @Override
3      public Profesor crearProfesor() {
4          return new Profesor.ProfesorBuilder()
5              .setNombre(nombre: "Profe de Fisica")
6              .setMateria(materia: "Fisica Mecanica")
7              .setVidaMaxima(vidaMaxima: 200)
8              .build();
9      }
10 }
```

Así se puede agregar otro tipo de profesor mediante otra fábrica. Sin embargo, todavía es necesario modificar el switch de Main para agregarlo al menú:

- **3. LSP — Sustitución de Liskov Cumple**

Este principio establece que las clases que heredan de otra deben poder sustituirla sin afectar el funcionamiento del programa.

En Main se utiliza:

```
73  ProfesorFactory profesorFactory = null;
```

Y esta variable puede recibir diferentes fábricas:

```
84  profesorFactory = new ProfesorCalculoFactory();
85  break;

87  profesorFactory = new ProfesorProgramacionFactory();
88  break;

90  profesorFactory = new ProfesorFisicaFactory();
91  break;
```

todas heredan de ProfesorFactory y utilizan el mismo método:

```
2  public abstract Profesor crearProfesor();
3  }
```

Por lo tanto, cualquiera de estas fábricas puede sustituir a ProfesorFactory y el programa continúa funcionando.

- **4. ISP — Segregación de Interfaces** **No cumple**

Este principio busca evitar interfaces demasiado grandes que obliguen a una clase a implementar métodos que realmente no necesita.

En el proyecto no tenemos una interfaz grande de ese tipo. Las clases no están obligadas a implementar métodos innecesarios.

Por ejemplo, no existe una interfaz con muchos métodos como:

```
interface Juego {  
    void atacar();  
    void guardar();  
    void imprimir();  
    void conectar();  
}
```

Por eso no se identifica una violación de ISP.

- **5. DIP — Inversión de Dependencias** **No cumple**

Este principio busca que las clases principales dependan de abstracciones y no directamente de clases concretas.

En Main se crean directamente las fábricas:

```
83 | | | | | case 1:  
84 | | | | |     profesorFactory = new ProfesorCalculoFactory();  
85 | | | | |     break;  
86 | | | | | case 2:  
87 | | | | |     profesorFactory = new ProfesorProgramacionFactory();  
88 | | | | |     break;  
89 | | | | | case 3:  
90 | | | | |     profesorFactory = new ProfesorFisicaFactory();  
91 | | | | |     break;
```

También se crea directamente:

```
22 | | | |  
23 | | | |     Partida partida = new Partida(profesor);
```

Esto significa que Main conoce y depende directamente de esas implementaciones concretas.

5. Selección de Patrones Creacionales

Factory Method (Método Fábrica) Patrón Seleccionado

Se eligió este patrón porque permite abstraer la creación de los distintos profesores y armas del juego. En lugar de usar new de forma directa dentro del controlador principal lo que aplicaría la lógica del juego a clases concretas, la creación de objetos se delega a clases fábrica específicas. Esto hace que agregar un nuevo tipo de profesor sea cuestión de crear una clase nueva, sin tocar el resto de la aplicación.

Singleton (Instancia Única)

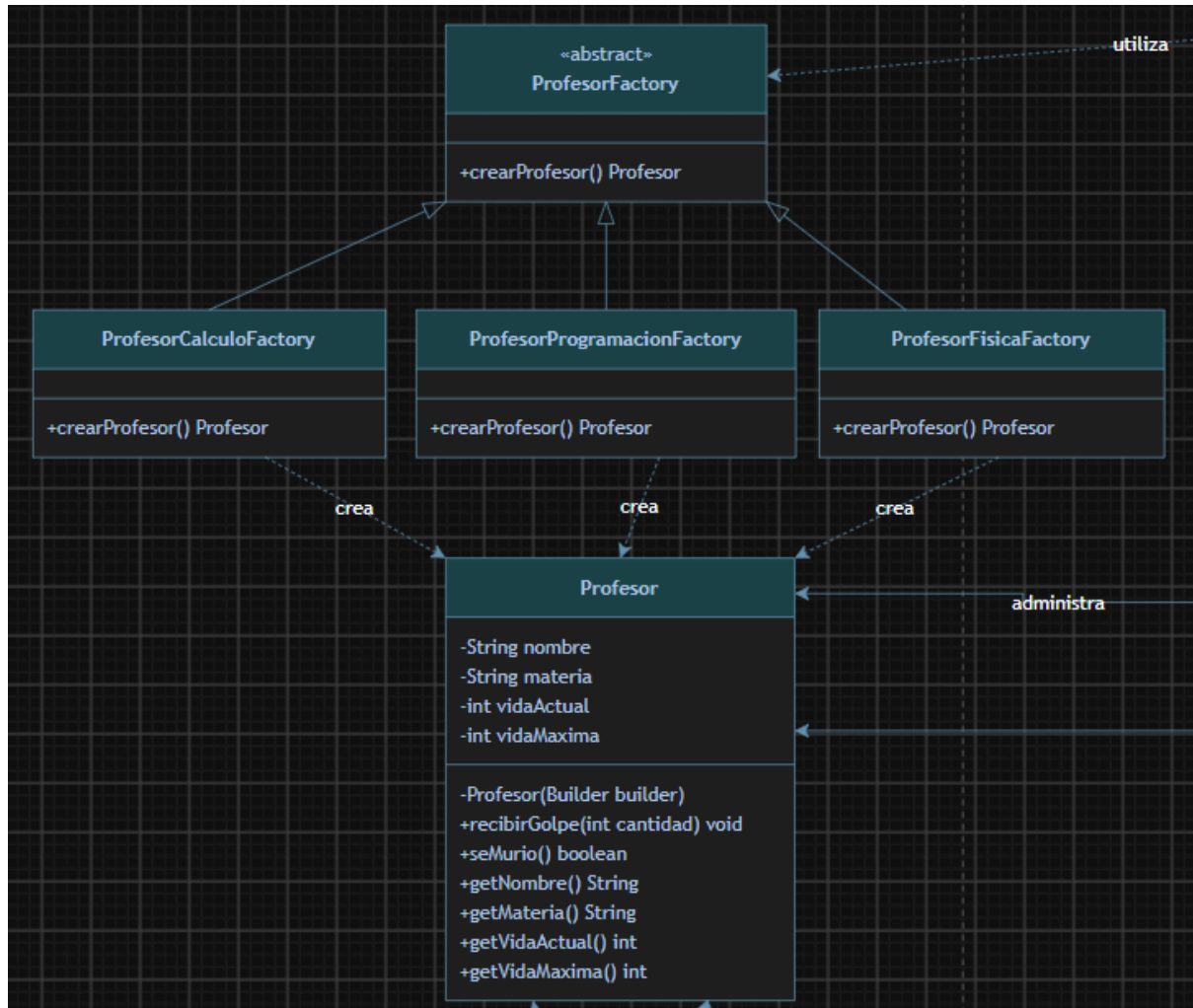
Se consideró para el gestor general de la partida (GameManager), ya que este componente debe existir como una única instancia durante todo el juego. Usar Singleton evita que se creen controladores duplicados en memoria y centraliza el estado de la partida en un solo punto.

Builder (Constructor)

Análisis: Permite construir objetos complejos paso a paso. Aunque no es el patrón principal en esta fase, resultaría muy útil si en futuras versiones los profesores se crean combinando diferentes accesorios, diálogos, niveles de armadura y habilidades especiales compuestas.

Factory Method (Método Fábrica)

Diagrama



Implementación:

```
public abstract class ProfesorFactory {

    public abstract Profesor crearProfesor();

}

public class ProfesorCalculoFactory extends ProfesorFactory {
```

```
@Override

public Profesor crearProfesor() {

    return new Profesor.ProfesorBuilder()

        .setNombre("Profe de Calculo")

        .setMateria("Calculo Diferencial")

        .setVidaMaxima(150)

        .build();

}

}

public class ProfesorFisicaFactory extends ProfesorFactory {

    @Override

    public Profesor crearProfesor() {

        return new Profesor.ProfesorBuilder()

            .setNombre("Profe de Fisica")

            .setMateria("Fisica Mecanica")

            .setVidaMaxima(200)

            .build();

    }

}

public class ProfesorProgramacionFactory extends ProfesorFactory {

    @Override

    public Profesor crearProfesor() {

        return new Profesor.ProfesorBuilder()
```

```

        .setNombre("Profe de Programacion")

        .setMateria("POO")

        .setVidaMaxima(100)

        .build();
    }
}

```

Uso en main:

```

ProfesorFactory profesorFactory = null;

while (profesorFactory == null) {

    System.out.println("\n--- VICTIMAS ---");

    System.out.println("1. Profe de Calculo (Calculo
Diferencial - Salud: 150)");

    System.out.println("2. Profe de Programacion (POO - Salud:
100)");

    System.out.println("3. Profe de Fisica (Fisica Mecanica -
Salud: 200)");

    System.out.print("\nElige tu victima: ");

    try {

        int opcionProfe = Integer.parseInt(scanner.nextLine());

        switch (opcionProfe) {

            case 1:

                profesorFactory = new ProfesorCalculoFactory();

                break;

            case 2:

```

```

        profesorFactory = new
ProfesorProgramacionFactory();

        break;

    case 3:

        profesorFactory = new ProfesorFisicaFactory();

        break;

    default:

        System.out.println("Error: Opcion invalida.
Intenta de nuevo.");

    }

    } catch (NumberFormatException e) {

        System.out.println("Error: Por favor ingresa un numero
valido.");

    }

}

return profesorFactory;

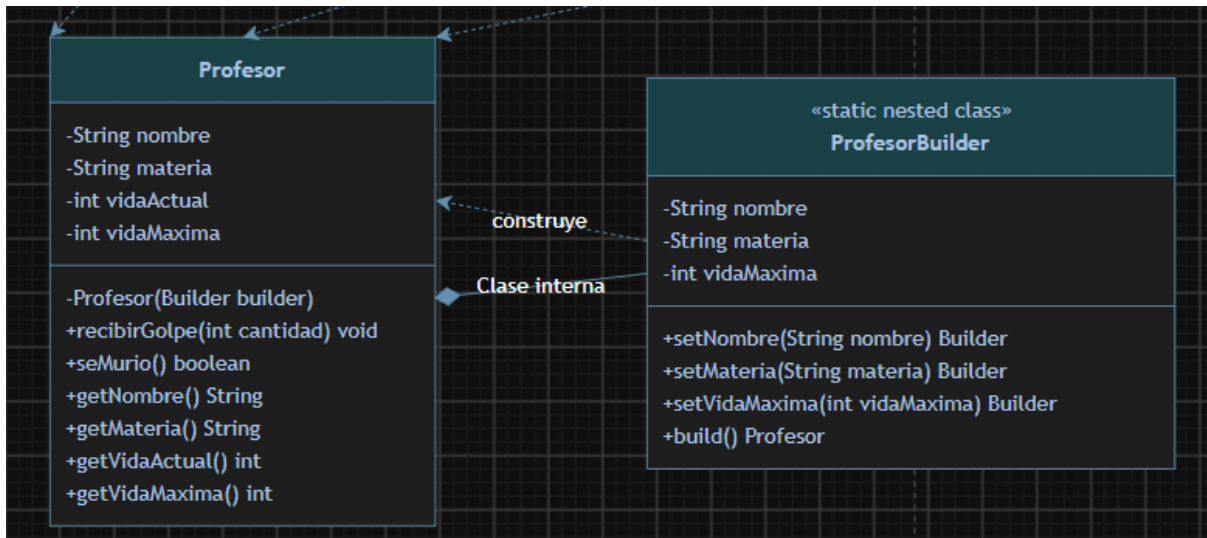
}

```

El patrón Factory Method permite encapsular la creación de los profesores. ProfesorFactory define el método crearProfesor(), mientras que cada fábrica concreta determina qué configuración de profesor se debe crear. De esta manera, la creación de objetos queda separada de la lógica principal del juego.

BUILDER

Diagrama:



Implementacion:

```
public class Profesor {

    private String nombre;

    private String materia;

    private int vidaActual;

    private int vidaMaxima;

    private Profesor(ProfesorBuilder builder) {

        this.nombre = builder.nombre;

        this.materia = builder.materia;

        this.vidaMaxima = builder.vidaMaxima;

        this.vidaActual = builder.vidaMaxima;

    }

    public static class ProfesorBuilder {
```

```
private String nombre;

private String materia;

private int vidaMaxima;


public ProfesorBuilder setNombre(String nombre) {

    this.nombre = nombre;

    return this;
}


public ProfesorBuilder setMateria(String materia) {

    this.materia = materia;

    return this;
}


public ProfesorBuilder setVidaMaxima(int vidaMaxima) {

    this.vidaMaxima = vidaMaxima;

    return this;
}


public Profesor build() {

    return new Profesor(this);
}

}


public void recibirGolpe(int cantidad) {
```

```
        this.vidaActual -= cantidad;

        if (this.vidaActual < 0) {

            this.vidaActual = 0;

        }

    }
}
```

```
public boolean seMurio() {

    return this.vidaActual <= 0;

}
```

```
public String getNombre() {

    return nombre;

}
```

```
public String getMateria() {

    return materia;

}
```

```
public int getVidaActual() {

    return vidaActual;

}
```

```
public int getVidaMaxima() {

    return vidaMaxima;

}
```

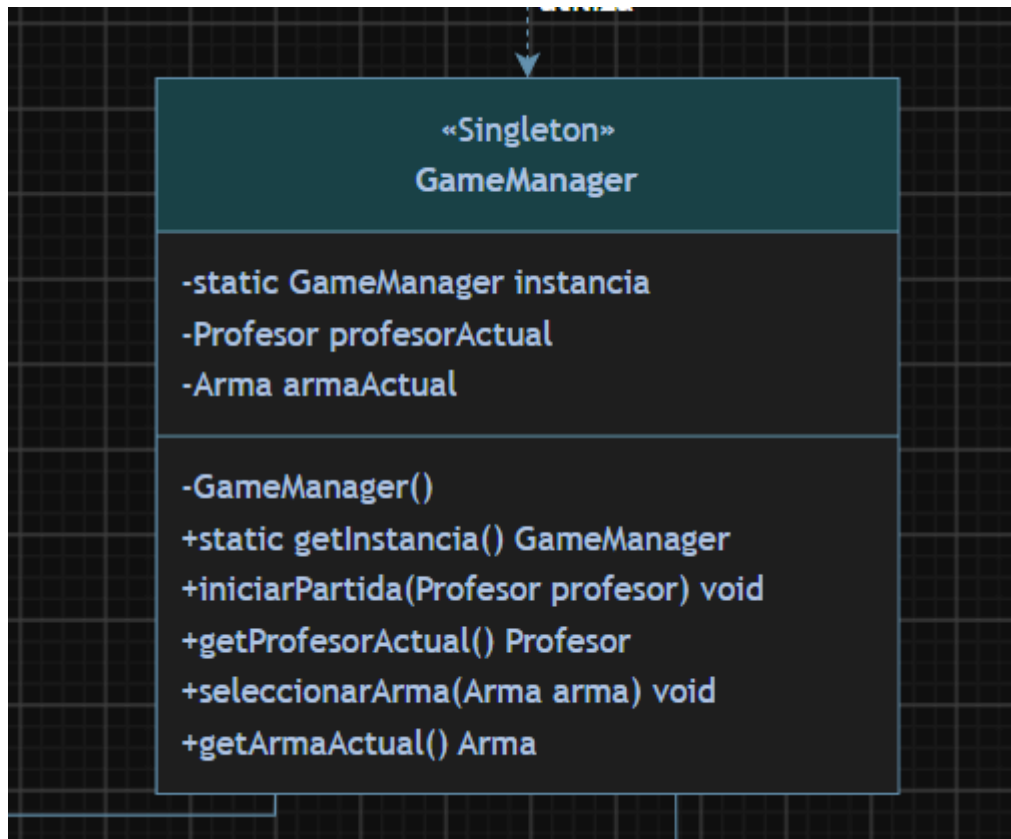
```
    }  
}
```

Uso:

```
public Profesor crearProfesor() {  
  
    return new Profesor.ProfesorBuilder()  
  
        .setNombre("Profe de Fisica")  
  
        .setMateria("Fisica Mecanica")  
  
        .setVidaMaxima(200)  
  
        .build();  
  
}
```

El patrón Builder permite construir un objeto Profesor paso a paso mediante diferentes métodos de configuración. En lugar de utilizar un constructor con todos los parámetros directamente, se establecen el nombre, la materia y la vida máxima antes de ejecutar build(), que finalmente crea el objeto Profesor.

SINGLETON Diagrama:



Implementación:

```
public class GameManager {

    private static GameManager instancia;

    private Profesor profesorActual;

    private Arma armaActual;

    private GameManager() {

    }

    public static GameManager getInstance() {

        if (instancia == null) {

            instancia = new GameManager();

        }

    }

}
```

```

    }

    return instancia;
}

public void iniciarPartida(Profesor profesor) {

    this.profesorActual = profesor;
}

public Profesor getProfesorActual() {

    return profesorActual;
}

public void seleccionarArma(Arma arma) {

    this.armaActual = arma;
}

public Arma getArmaActual() {

    return armaActual;
}
}

```

Uso en Main:

```

gameManager = GameManager.getInstance();

luego:

gameManager.iniciarPartida(profesor);

```

luego:

```
gameManager.seleccionarArma(armaElegida);
```

El patrón Singleton garantiza que GameManager tenga una única instancia durante la ejecución del programa. Esto permite centralizar el estado principal de la partida, como el profesor actual y el arma seleccionada. El constructor es privado y la instancia se obtiene mediante el método estático `getInstancia()`.